

Paddle 库源码解读

陈飞宇 2021/05/15

首先来读一读 paddle 的 Tensor,亦即数组库。

paddle::framework 命名空间

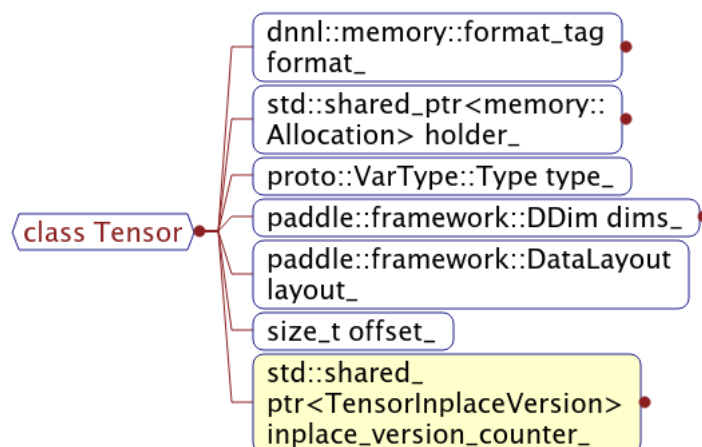
这里主要是一些框架底层设计相关的类和函数。我们暂且不涉及具体的 operator, 而是探索 paddle 框架本身的设计。主要分几个部分:

1. Tensor 相关的,主要是 Tensor 作为一个数组库的设计,里面会有一个数组库该有的功能,还包含 Tensor 的 metadata 相关的类;
2. Variable 相关的,这里 Variable 是对一般编程概念上变量的抽象,提供了描述程序的基本概念;
3. Scope 相关的,Scope 是对一般编程意义上作用域的抽象,这是和变量管理的类。
4. Attribute 相关的,这是为了准备 Operator 功能的一部分,类似一个 argparse 一样的东西;
5. Operator 相关的。包含各种 Context。以及 Kernel 相关的类。OpMaker 相关的类。Operator 注册相关的。
6. Executor 相关的。

paddle::ramework::Tensor

文件位置 projects/Paddle/paddle/fluid/framework/tensor.h

首先看数据成员



Tensor 里面包含了一个指向存储的指针 holder_, scalar 类型标识 type_, 形状 dims_, 偏移量 offset_. 这些是一个数组类型一般来说应该有的属性。

另外一些需要提及的是数据布局 layout_, 而且在有 mkldnn 的情况下还有一个 format_, 这两个貌似是有一些功能上的重复。format_ 是为了 mkldnn 的 ad-hoc 设计, 而且看起来是后加的设计。

另外一个引用计数 inplace_version_counter_, 这个东西终于加入了。此前的版本是没有这个的。

需要注意一点:

Tensor 不是像 Eigen 中的 Tensor 那样的设计, 它没有一个 typename 作为模板参数, 而且, 它也不是一个类模板, 而是一个类。它用一个数据成员来存储自己的类型, 亦即 type_, 这个是和 proto 相关的, 参见 [paddle::framework::proto::VarType::Type](#)。但是在很多函数中, 还是有一个模板参数是 typename。(前者是用来处理 protobuf 的, 而后者才是 c++ 中的类型。

这里引入了两层抽象。一层是 Allocation, 是数据所在的内存区域的表示; 另一层是 Tensor, 里面还有形状, 类型之类的概念。

paddle::framework::LoDTensor

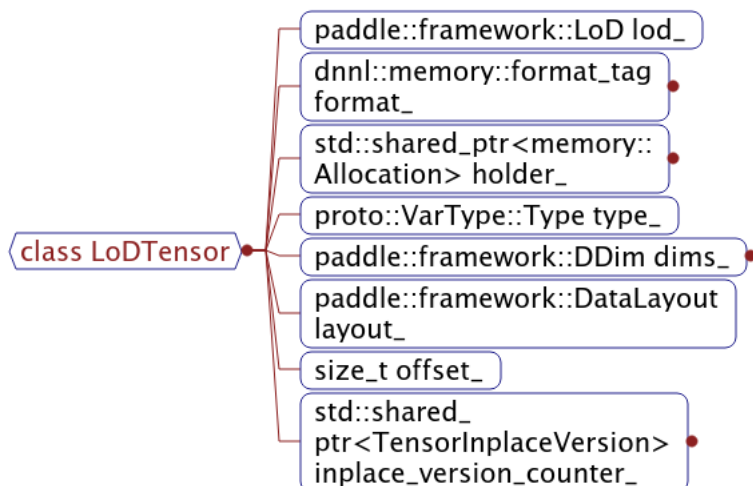
文件位置 projects/Paddle/paddle/fluid/framework/lod_tensor.h

是对 Tensor 的一种扩展。它是可以不整齐的。可以说是 nested list 的一种抽象。它比 Tensor 多了偏移量记录。这里我们可能会有一个问题: “既然 Tensor 都不整齐了, 那么 Tensor 的形状是不是就没法定义了? LoDTensor 继承 Tensor 是否是不合适的设计?”

一个解释可以是这样的。即使我们认为 LoDTensor 继承 Tensor 不合适, 那么至少它可以继承 1D Tensor 吧? 这点还是可以得到共识的。嵌套的列表都可以展平。展平后, 里面的元素类型都是 Tensor 的 scalar type 类型的。在 C++ 内存模型的基础上, 一般意义上的线性容器类型会要求这个类型是一个 POD 类型, 这样才能把它装进线性容器里。

那么我们提及一个概念, 如果这个 type 也可以是有内部结构的呢? 假设它是一个 super-scalar, 比如说都是一个 (3,3,3) 形状的魔方。那么由它们构成的 nested list 也可以展开为魔方的 1D list。与此类似, LoD Tensor 的形状其实就是这么定义, 前面所有嵌套展平的长度, L, 后面是 super-scalar 的形状。所以 LoD Tensor 的形状是 (L, *base_shape)。

我们先来看一下数据成员图



可见只是比 Tensor 多了 lod_ 这个数据成员。而 LoD 设计成了 `vec<vec<int>>`, 有多少层 LoD 就有多少个子 vec, 一层里有 n 段, 就会有 n+1 个端点。

paddle::framework::LoDTensorArray

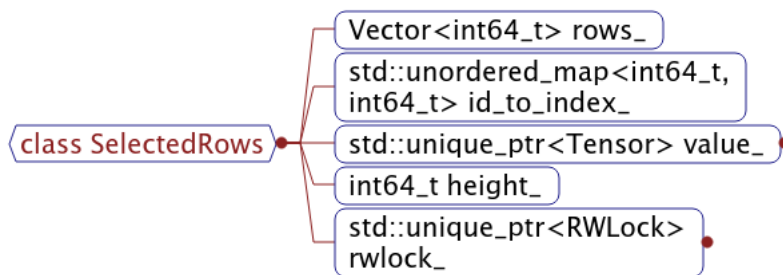
文件位置 projects/Paddle/paddle/fluid/framework/lod_tensor_array.h

其实它就是一个 using 类型别名。就是 std::vector of LoDTensors.

paddle::framework::SelectedRows

文件位置 projects/Paddle/paddle/fluid/framework/selected_rows.h

SelectedRows 是稀疏 tensor 的一种简化的表示。数据成员图如下:



value_是一个指向 Tensor 的指针,rows_是行号。height_向量的个数。id_to_index_是映射表,是 key 到行号的映射关系。rwlock_是一个文件锁。看起来这就是用作 distributed lookup table 的表的类型。

paddle::framework::Array

文件位置 projects/Paddle/paddle/fluid/framework/array.h

为了有一个通用的能在 CPU 和 GPU 上使用的静态数组而设计的类模板。功能上和 std::Array 类似。不过那些需要遍历的操作都用模板和宏展开了。以追求更高的性能。带有两个模板参数 **typename T**, **size_t N**。

数据成员图如下,其中的 kSize 是一个静态变量。



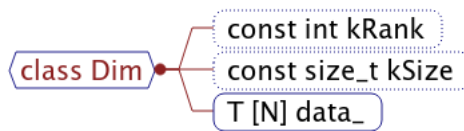
同时还有一个零长度特化。没有数据存储区。



paddle::framework::Dim

文件位置 projects/Paddle/paddle/fluid/framework/dim.h

paddle 中的形状类模板,有一个模板参数 **int D**。继承了 **Array<int64_t, D>**,还有一个静态变量,就等于 D。也就是维度个数 Rank.

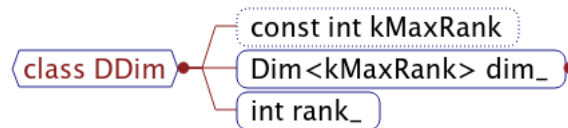


静态变量并不属于实例的一部分,因此 Dim 对象的内存也就只有 data_而已。这个类似 Eigen 中的 Eigen::Sizes.

paddle::framework::DDim

文件位置 projects/Paddle/paddle/fluid/framework/ddim.h

paddle 中的动态形状类。类似 Eigen 中的 Eigen::DSizes. 由于 paddle 框架的限制,目前只是支持 0d-9d 的操作,而且对于 0d 的支持并不是健全的。数据成员图如下:



其中 kMaxRank 就是 9,然后成员包含 rank,以及一个 rank 阶的 Dim.真是有趣,动态形状里存了一个静态形状。值得注意的是:它不再是一个类模板,而是一个类。

paddle::framework::LoD

文件位置 projects/Paddle/paddle/fluid/framework/lod_tensor.h

这个其实只是 `std::vector<Vector<size_t>>` 的别名。只要符合某些特征就可以是一个 **LoD**,为什么不做成一个正经类型呢?可能是一开始觉得没有必要吧。

第三层	0									1											2
第二层	0				1					2											3
第一层	0	1			2		3		4		5			6					7		
super-scalar	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

比如说一个 3 层的 LoD

```
[[0, 2, 3],
 [0, 2, 4, 7],
 [0, 2, 5, 7, 10, 12, 15, 20]]
```

它可以这么解释,第三层有三个端点,说明有两个序列。其中第一个序列包含两个子序列,第二个序列包含一个子序列。(参见途中的第三层。这里我们用的次序和 LoD 中的次序相反,最顶层次序最大)

第二层有 3 个序列。(这也可以从第三层中观察出来,因为第三层的两个子序列分别包含 2, 1 个子序列)。故有 4 个端点。3 个序列分别含有 2, 2, 3 个子序列。

第一层有 7 个序列。这也可以从上一层中三个序列分别包含 2, 2, 3 个子序列计算出来。故包含 8 个端点。每个序列的长度分别是 2, 3, 2, 3, 2, 3, 5。

这个设计其实很无敌的啦。不过叫做 recursively-packed tensor 或者更加恰当。总之就是一个 1D Tensor 加上分隔信息构成的。但是我们日常也就只用 LoD level=1 的 LoDTensor,亦即只有一层分隔的 Tensor。

所以后来看这个就总是觉得有点割鸡用牛刀。

paddle::framework::DataLayout

文件位置 projects/Paddle/paddle/fluid/framework/data_layout.h

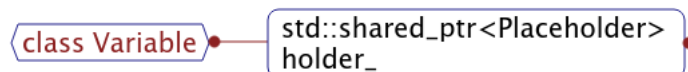
DataLayout 是一个枚举类型。kKHW 和 kNCHW 是和 CUDA 相关的,没关系的用 kAnyLayout, kMKLDNN 是 MKLDNN 专用的。

这个也就只是一个标识符。用于程序区分具体的行为。里面还附带了 DataLayout 和字符串之间的互相转换的函数。

paddle::framework::Variable

文件位置 projects/Paddle/paddle/fluid/framework/variable.h

paddle 内部的一种变量类型(可以类比编程语言中的变量,当然也能有各种类型)。它更像一个包装,它能装任何东西。它装的东西的类型不是它的模板参数,而且它也不是类模型,而是一个类。它只持有一个成员,就是一个指向其包含的东西的指针。是一个经典的 pointer to implementation 的设计。



真就是一个变量。

paddle::framework::Variable::Placeholder

文件位置 projects/Paddle/paddle/fluid/framework/variable.h

这是 Variable 内部的一种 private 的类型。它更像是 Variable 的蜕皮的版本。其包含一个自由的指针,以及一个类型标识。



paddle::framework::Variable::PlaceholderImpl

文件位置 projects/Paddle/paddle/fluid/framework/variable.h

这个同样也是 Variable 内部的一种 private 类模板,它继承了 Placeholder,但是它是有一个类型作为模板参数。而 Placeholder 是持有一个类型标识作为成员变量的。那么总有一个什么东西作为桥梁,沟通类型和作为变量的类型标识。基本上就又是一个枚举类型,或者一个映射把它们的关系存起来了。

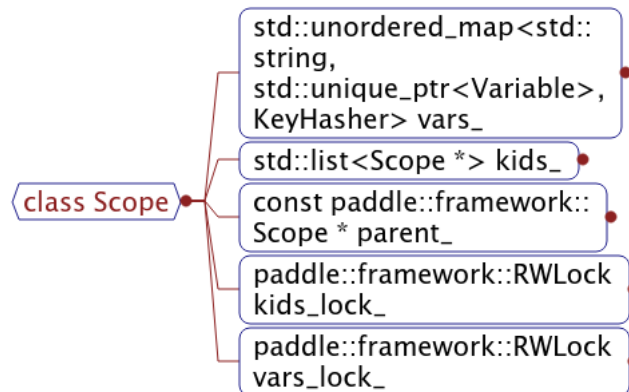


PlaceholderImpl 因为有了真正意义上的类型能够,所以可以持有一个额外的成员变量,亦即其指针指向的对象。而 Placeholder 则不能,因为它没有那个指针指向的东西的真正类型。

那么维持类型到一个类型标识(成员变量 `int type_`)之间的关系的是什么呢?

paddle::framework::Scope

scope 名字和 Variable 的映射。Variable 必须存在于某个 Scope 中。(作用域)。Scope 也可以嵌套。



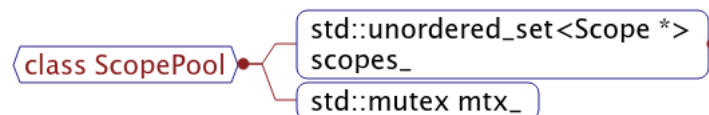
同时还有锁。这些锁是用于 Scope 的读写,开启子 Scope 时要加锁,创建新的变量时也要加锁。但是在 inference 的时候因为不会同时有两个线程写 Scope,所以 inference 的时候锁是无效的。

另外,Scope 也是一种不可复制(构造和赋值)的类型,因为里面有那么多 unique pointer, 所以自然也没法 copy。

paddle::framework::ScopePool

文件位置 `projects/Paddle/paddle/fluid/framework/scope_pool.h`

这个对象是一个 Scope 的聚集之处。其设计是为了管理所有的 Scope, 它被设计成一个单例模式的类。其数据成员如下:



包含一个 `Scope*` 的无序集合。并且含有一个互斥锁。

它被设计成单例的方式是,构造函数是私有的。无法从外部调用。而把构造的方法留在了静态方法 `Instance`. 在 `Instance` 里面创建一个 `pool`,并且返回。值得注意的是,这里使用了 `static` 关键字来延长局部变量的生存期,使得它不会被析构。这是在 c++ 下实现单例的常用手段之一。

```
ScopePool &ScopePool::Instance() { // NOLINT
    static ScopePool pool;
    return pool;
}
```

而 `ScopePool` 的 `Insert` 也实现了移动语义。使得作为 `prvalue`(纯右值)和 `xvalue`(将亡值)的 `scope` 可以移动到无序集合中被管理。(而且值得注意的是,`Insert` 的参数是 `std::unique_ptr<Scope> &&`, 本身也不支持复制。

```
void ScopePool::Insert(std::unique_ptr<Scope> &&s) {
    std::lock_guard<std::mutex> guard(mtx_);
    scopes_.insert(s.release());
}
```


而且我们前面的疑问,那些互斥锁是干什么用的,可以得到一定程度的解答,Scope 移动到 Pool 里面以及 Pool 的内容变动都是需要加锁的。

paddle::framework::VarTypeTrait

文件位置 projects/Paddle/paddle/fluid/framework/var_type_traits.h

这个就是沟通类型和类型标识之间的关系的 trait. 只有被注册了的类型才能拥有相应的 trait. 而注册机制是把这些放到一个联合类型中。这个联合类型我们后面会提到。它就是 `VarTypeRegistryImpl`. 在这里我们首先关注 `VarTypeTrait`.

它是一个类模板,里面有一个静态变量,就是这个类型对应的整数标识。

```
struct VarTypeTrait {
    const int kId;
};
```

paddle::framework::VarTypeRegistry

文件位置 projects/Paddle/paddle/fluid/framework/var_type_traits.h

这就是上面所说的那个联合类型。它是很多种类型的联合类型,它是 `paddle::framework::detail::VarTypeRegistryImpl` 的一个具体化。它同时还提供了查询某个具体类型在这个联合类型中的位置的功能。它包含一个静态变量,是包含的类型个数。

```
struct VarTypeRegistryImpl {
    const size_t kRegisteredTypeNum;
};
```

不过值得注意的是,这里使用了一种松散的方式来组织这种关系。首先有一个联合类型。但是并没有一个字典来把类型对应到整数,而是由各个 `trait<T>` 类型来决定类型 T 对应的 id。然后文件中提供了一个宏去特化 `VarTypeTrait` 以使得这些类型对应的 id 是我们想要的。其他的类型的 id 则是根据在联合类型中的配置查询出来的。为了避免冲突,来做了一些偏移量。

paddle::framework::Attribute

文件位置 projects/Paddle/paddle/fluid/framework/type_defs.h

`Attribute` 是一个 `boost::variant` 是具体化。包含了 14 种类型,但是没有 `double`. 奇怪!

paddle::framework::AttributeMap

文件位置 projects/Paddle/paddle/fluid/framework/type_defs.h

就是 `string → Attribute` 的无序映射容器。

以及这个文件里还有华为升腾专用的 `NPUAttribute` 和 `NPUAttributeMap`.

paddle::framework::ExtractAttribute

文件位置 projects/Paddle/paddle/fluid/framework/attribute.h

这是 Attribute 提取器的类模板。其模板参数是类型 `typename T`, 提取到的属性会被处理成这个类型的。

每个属性提取器有自己的名字。应该是和 AttributeMap 的设计配合使用的, 提取到了就存到 Map 里面。基类是带有 `operator()` 的, 因此都是 functor. 数据成员图如下, 这个类及其子类都是功能为主的 functor, 数据成员都只有名字。



paddle::framework::AttrReader

文件位置 `projects/Paddle/paddle/fluid/framework/attribute.h`

这个也是一个和 attribute 解析有关的类。

其数据成员图如下:



亦即, 里面仅仅存了一个 `str → Attribute` 的映射。它的初始化需要一个 attribute map. 然后它有一个 Get 方法, 这个方法需要一个类型参数 `typename T`, 因此 AttrReader 也只是一个很虚的存在。它本身并不知道 attribute map 里面存的东西的实际类型, 还要在 Get 的时候提供类型。

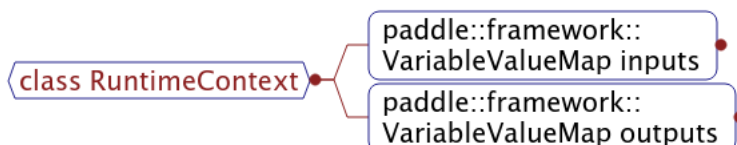
所以这里实际上好多个类做了一件事。把各种类型的东西存在一个类似 `str → Any` 的容器里, 然后按照类型将其取出。这个事情其实非常类似 `argparse` 或者 `boost::program_options` 的功能, 因为都是要处理 `str → Any` 的映射。

下面我们来读和 operator 相关的代码。

paddle::framework::RuntimeContext

文件位置 `projects/Paddle/paddle/fluid/framework/operator.h`

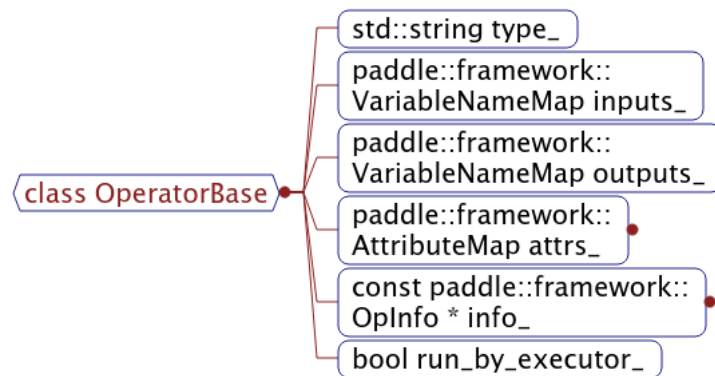
这是运行上下文, 它在一个 Scope 内。且记录了 input Variables 和 output Variables.



paddle::framework::OperatorBase

文件位置 `projects/Paddle/paddle/fluid/framework/operator.h`

paddle 中所有 operator 的基类。数据成员图如下:



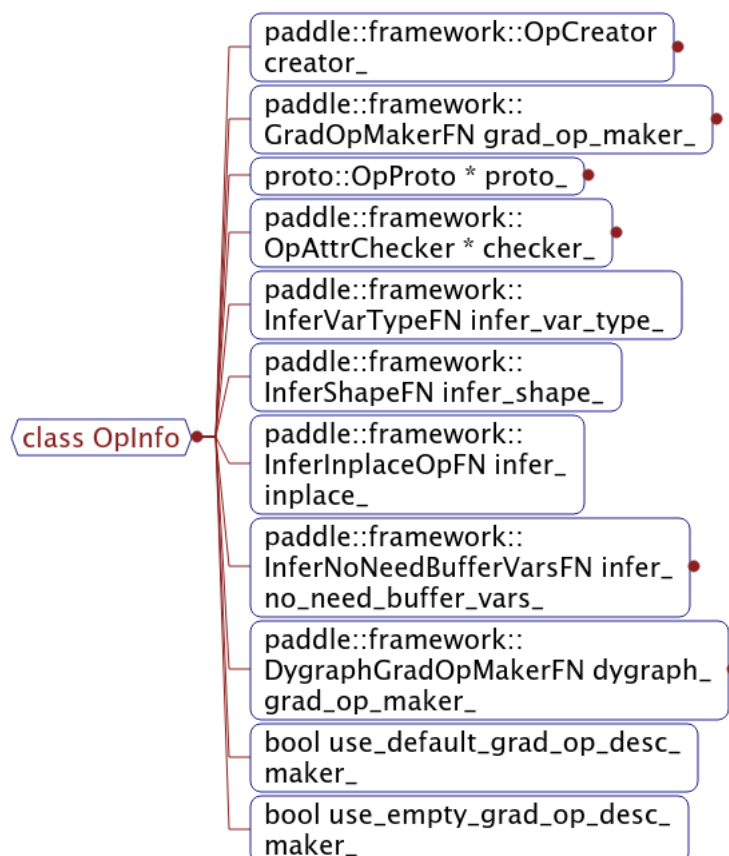
operator 会记录自己的输入输出,以及 attribute. 而 `attrs_` 是一个名字到 Attribute 的映射。

同时 `type_` 是这个 operator 的名字。`run_by_executor` 估计是和动态图相关的功能,表示它由 executor 运行还是 tracer 运行。那么 `OpInfo` 是什么东西?记录了很多东西。

paddle::framework::OpInfo

文件位置 `projects/Paddle/paddle/fluid/framework/op_info.h`

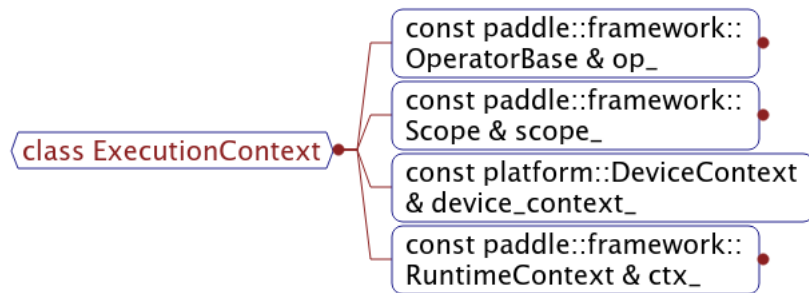
和 op 有关的信息的记录。内容好多。包括用于推断输出的形状,设备的功能, `GradOpMaker` 功能,Op 的 proto 信息。推断是否 inplace 的功能,推断是否需要 buffer 的功能,还有一些 flag 用于指定是否使用默认 `grad_op_desc_maker` 等。



paddle::framework::ExecutionContext

文件位置 projects/Paddle/paddle/fluid/framework/operator.h

执行上下文这又是个什么东西?首先看其数据成员



包含一个 `operatorbase`, 同时有一个设备上下文,和运行时上下文。还有一个 `scope`(作用域).啊~设备上下文参见 [DeviceContext](#).

那么 `Scope` 又是什么呢?

paddle::framework::VariableNameMap

文件位置 projects/Paddle/paddle/fluid/framework/type_defs.h

就是一个 `str → vec<str>` 的 typedef.

paddle::framework::VariableValueMap

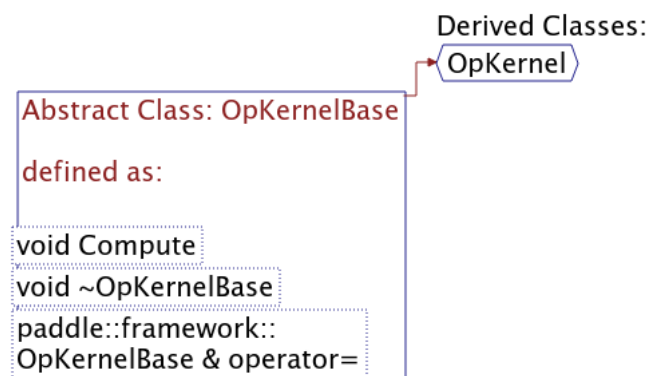
文件位置 projects/Paddle/paddle/fluid/framework/type_defs.h

就是一个 `str → vec<paddle::framework::Variable*>` 的 typedef.

paddle::framework::OpKernelBase

文件位置 projects/Paddle/paddle/fluid/framework/operator.h

这是 `OpKernel` 的基类,亦即我们常说的 `Kernel`.它没有数据成员,只是提供了一套接口。其中说明 `OpKernel` 有一个 `Compute` 方法,而且这个方法接受一个 `ExecutionContext` 作为其唯一的参数。



然后最常见的一个做法是 Kernel 以一个类型为模板参数可以生成几个具体化的版本。所以就有了 OpKernel 类模板。

paddle::framework::OpKernel

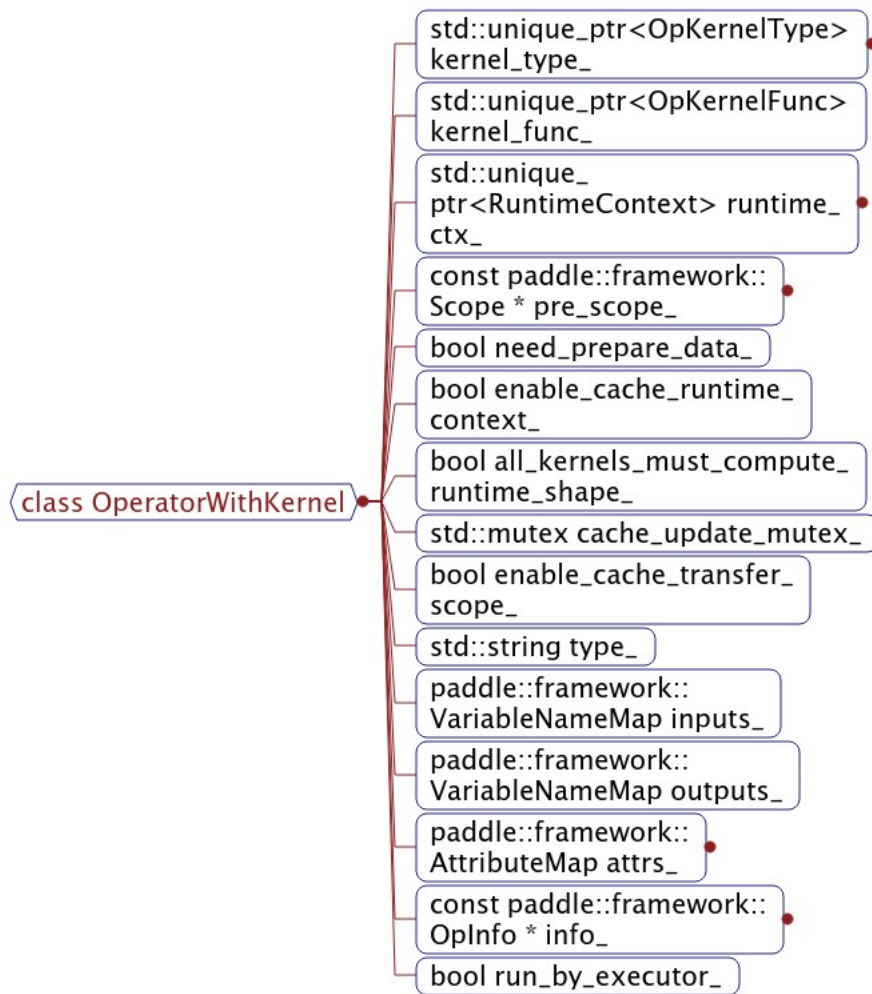
文件位置 projects/Paddle/paddle/fluid/framework/operator.h

这是符合 OpKernelBase 接口规范的一个类模板。带有一个模板参数 typename T.

paddle::framework::OperatorWithKernel

文件位置 projects/Paddle/paddle/fluid/framework/operator.h

这是所有带有 Kernel 的 Op 的基类。它本身也以 OperatorBase 为基类。数据成员图如下：



其中比较重要的几个成员：

1. `kernel_type_` 是一个 `KernelType` 实例,这里包含了数据类型,设备,库类型等几个信息,这共同描述了一个 `Kernel` 的类型。
2. `kernel_func` 就是真正的可执行对象。它的签名都是接受一个 `ExecutionContext` 对象,并且没有返回值的,因此,请自备 `outputs`。
3. `runtime_ctx_` 这个是运行时上下文。(这个东西的语义我们弄清楚再来细讲。)
4. `pre_scope_`是这个 `op` 运行的 `scope`。
5. `inputs_`, `outputs_`, `attrs` 是一个 `op` 运行所需要的外部输入的信息,[AttributesMap](#) 我们前面已经讲过。但是 `VariableNameMap` 是什么的,其实就是一个映射的 `typedef`。

其中的不少数据成员都是从基类 `OperatorBase` 中带来的。

其他的数据成员,我们后面再细说。

paddle::framework::OpProtoAndCheckerMaker

paddle::framework::Executor

paddle::framework::proto 命名空间

这个命名空间主要是和 protobuf 有关的东西。主要是为了给 python 端提供接口用的。Program, Block, Variable, Param 等都有自己的 Desc.而这个命名空间的类基本都依赖一个编译过程中生成的巨大头文件 projects/Paddle/build/paddle/fluid/framework/framework.pb.h.

paddle::framework::proto::VarType::Type

文件位置 projects/Paddle/build/paddle/fluid/framework/framework.pb.h 注意这个文件是编译的过程中生成的。和编译之前的 projects/Paddle/paddle/fluid/framework/framework.proto 有密切关系。生成的 framework.pb.h 一共 5000 多行,实在有点恐怖。

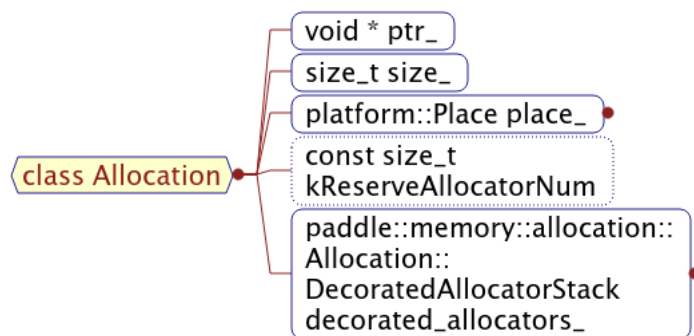
这里定义了各种类型。不仅包括 Tensor 的 scalar 类型,还有很多特别的类型比如 feeedict, fetchlist,raw 之类的类型。proto 还是一个我不太理解的东西。总是它也是一种枚举类型。

paddle::memory 命名空间

paddle::memory::allocation::Allocation

文件位置 projects/Paddle/paddle/fluid/memory/allocation/allocator.h

paddle::memory::Allocation 是对一片数据区域的表示。与之相关是一个概念是 Allocator,Allocation 是 Allocator 分配的产物。首先看其数据成员,包含一个特别自由的指针 void*指向内存区域,还有内存区域的大小,单位是 bytes,所以它基本上完全是 buffer 区。还有设备标识。



但是那个 decorated_allocators 又是什么呢?根据源码里面的注释。内存分配里存在 retry 机制,如果第一个 allocator 分配不成功,第二个可能成功,那么得到分配的 Allocation 需要记录是谁成功分配了它,也可以不记录,就直接反序释放即可。

所以就使用了一个 vector 来记录 Allocator,然后由于性能的原因,这里就永乐一种特别的 InlinedVector. (这个后续我们也可以看看为什么要这么做。)

其实从 Raw 指针到 Allocation 之间也有一层抽象,Allocation 可能可以修改自身的数据成员 ptr_,从而改变存储位置。但是可能只是用于去碎片化之类的操作,因为一层一层的语义有差别。Tensor 和 Allocation 之间的关系和 Allocation 与 raw pointer 之间的关系是不同的。

paddle::platform 命名空间

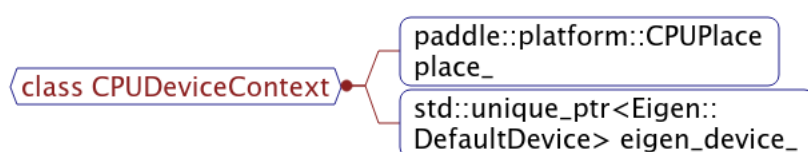
paddle::platform::DeviceContext

文件位置 projects/Paddle/paddle/fluid/platform/device_context.h

设备上下文。这是一个基类。它么有任何成员,仅提供接口。

paddle::platform::CPUTDeviceContext

这是 CPUTDeviceContext,是 CPU 上用的。包含一个 Place, 这是对设备之抽象。然后还有一个就是 Eigen device. 这是为了使用 Eigen 的功能而设计的。



其他的硬件也有对应的 DeviceContext,比如 CUDAContext,但是笔记本上编译的版本没有包含 nvidia 的 GPU 以及其他加速卡,所以这里不做展示了。

关于各种 Place。

paddle::platform::CPUTPlace

CPUTPlace 不包含任何数据成员。

paddle::platform::CUDAPlace

CUDAPlace 只含有设备号一个成员。



诸如此类。然后 Place 是这些 Place 的 variant 的子类。假如后续有更多的支持的设备,这个列表还会扩张。

python.paddle.fluid.framework 模块

python.paddle.fluid.framework.Program

python 端的 Program 是一个 python 端的类型。其 attribute 数量太多了。其重要的数据成员包含

1. desc 这是 core.ProgramDesc()返回的结果,而这也就是 ProgramDesc 类的实例。这个 pybind 类型直接绑定的是 paddle::framework::ProgramDesc,而它位于 projects/Paddle/paddle/fluid/framework/program_desc.h. 类似这种 python 端的类型包含一个对应 pybind 对象而不是直接就是这个 pybind 对象的模式还有不少。
2. blocks 这是一个 List[Block]类型的对象。

3. `current_block_index` 是一个整型变量,记录当前 Block 在 `blocks` 中的位置。

python.paddle.fluid.framework.Block

python 端的一个 Block 类。其主要的成员包含:

1. `desc`: 所在的 program 的 `ProgramDesc` 中的第 `idx` 个 block,事实上是一个 `paddle::framework::BlockDesc` 的引用(这也是 python 端的类型包含一个对应 `pybind` 对象);
2. `vars`: `OrderDict[st, Variable]` 的字典;
3. `ops`: `List[Operator]`;
4. `program` 指向它所在的 program 的引用。

python.paddle.fluid.framework.Operator

这也是一个在 python 端定义的类型。它的初始化方式如下。它需要的信息不少,但是在静态图和动态图下有不同的处理方式。动态图下仅需要 `type` 和 `attrs`,而静态图下需要 `block`,`desc` 等完整的信息。

```
def __init__(self,
             block,
             desc,
             type=None,
             inputs=None,
             outputs=None,
             attrs=None):
```

静态图下的执行方法基本上是构建一个 `opdesc`(根据这里的设计习惯,应该就是 `paddle::framework::OpDesc`),亦即 `self.desc`。根据 `type` 和 `inputs`, `outputs`, `attrs` 填充完 `self.desc` 的内容。

python.paddle.fluid.framework.Variable

这是 python 端的 `Variable`,和 C++端的 `Variable` 不是一个概念,和 `VarBase` 也不同。其初始化方式如下,虽然文档里面说这个类的实例不应由 `__init__` 方法来初始化,而应该用辅助方法。确切地说,静态图下用 `Block.create_var`,而动态图下创建的是 `VarBase` 也和它没有关系。

```
def __init__(self,
             block,
             type=core.VarDesc.VarType.LOD_TENSOR,
             name=None,
             shape=None,
             dtype=None,
             lod_level=None,
             capacity=None,
             persistable=None,
             error_clip=None,
             stop_gradient=False,
             is_data=False,
```

```
need_check_feed=False,  
belong_to_optimizer=False,  
**kwargs):
```

它同样也有一些重要的属性。

1. desc, 一个 `paddle::framework::VarDesc` 对象;
2. op 生产它的 op;
3. shape, dtype, lod_level 这些和形状,数据类型有关的 metadata;
4. is_data, 是否叶节点;
5. stop_gradient 是否阻断梯度回传。

值得注意的是, `__init__` 并不一定意味着创建, 它其实是面向字符串编程的, 如果这个名字在 block 中已经存在, 则返回已经存在的那个。

python.paddle.fluid.framework.Parameter

Variable 的子类。它额外有不少和训练相关的属性。比如 trainable, need_clip 之类的。

python.paddle.fluid.framework.NameScope

这是一个纯粹的 python 端的类型, 是为了产生 hierarchical 的名字而定义的。它产生了类似 model/linear, model/linear_1 这样的名字。

python.paddle.fluid.framework._Scope

这个是 c++ 端的 [Scope](#) 绑定到 python 端的类型。但是因为有一个下划线, 很明显, 设计者并不打算让 python 端的用户直接去使用它的初始化方法来初始化一个 Scope。而且 projects/Paddle/paddle/fluid/pybind/pybind.cc 中还定义了一个函数名字就叫 Scope, 它会负责创建并返回一个 Scope, 但是它额外做了一件事, 把这个新创建出来的 Scope 放到 pool 中。而这个 pool 是一个 singleton。这是为了管理 scope, 当 scope pool 析构的时候, 析构所有的 scope。

scope pool 是一个 `paddle::framework::ScopePool` 实例。可以参考前面的章节。

专题：Executor 是如何运行一个 Program 的

我们直接从 Executor 的 run 函数开始

projects/Paddle/python/paddle/fluid/executor.py

```
def run(self,  
        program=None,  
        feed=None,  
        fetch_list=None,
```

```

feed_var_name='feed',
fetch_var_name='fetch',
scope=None,
return_numpy=True,
use_program_cache=False,
return_merged=True,
use_prune=False):

```

它以一个 program 为参数,还有 feed 和 fetch_list,其他的部分可以暂且不管。

这会调用另一个类方法

```

return self._run_impl(
    program=program,
    feed=feed,
    fetch_list=fetch_list,
    feed_var_name=feed_var_name,
    fetch_var_name=fetch_var_name,
    scope=scope,
    return_numpy=return_numpy,
    use_program_cache=use_program_cache,
    use_prune=use_prune,
    return_merged=return_merged)

```

这两个函数基本没有差别。实现的方法在 _run_impl 里面。

```

def _run_impl(self, program, feed, fetch_list, feed_var_name,
              fetch_var_name, scope, return_numpy, use_program_cache,
              return_merged, use_prune):

```

其中 program 可以不传,如果不传,则使用默认的 main program, scope 也可以不传,那么就会使用 global_scope. 我们暂不考虑 CompiledProgram 和 rune_prune,而是考虑最简单的直接运行的方法。那么会调用

```

return self._run_program(
    program,
    feed=feed,
    fetch_list=fetch_list,
    feed_var_name=feed_var_name,
    fetch_var_name=fetch_var_name,
    scope=scope,
    return_numpy=return_numpy,
    use_program_cache=use_program_cache)

```

一般 feed 是一个字典,如果是在静态图下用多卡训练则是一个 list,其中装的都是 dict,一张卡对应一个。暂且 use_program_cache 的情形,那么首先做的事情是在图上添加 feed 和 fetch op,没错,这些也都是 op.

```

program = self._add_feed_fetch_ops(
    program=program,
    feed=feed,
    fetch_list=fetch_list,
    feed_var_name=feed_var_name,
    fetch_var_name=fetch_var_name)

```

我们首先看看这个添加 feed 和 fetch 的操作

```

tmp_program = program.clone()
global_block = tmp_program.global_block()

```

首先竟然就是直接 clone 一份。然后根据名字去创建获取或者创建 Variable,分别表示 feed_dict 和 fetch_list 在图上对应的 Variable(注意,整个 feed_dict 或者 fetch_list 都是一个 Variable,而不是说有多少个字段就有多少个,因此可见 Variable 能表示的东西是很多的). 然后添加 feed 和 fetch op.

添加 feed 的操作是

```

for i, name in enumerate(feed):
    if global_block.has_var(name):
        out = global_block.var(name)
        global_block._prepend_op(
            type='feed',
            inputs={'X': [feed_var]},
            outputs={'Out': [out]},
            attrs={'col': i})

```

可见是根据 feed 字典里面的 key 来查找变量,那么根据我们 feed 和 data 层的名字对应的关系,找到的就是 data 层的输出 Variable,然后把这个作为 feed_op 的输出。值得注意的是,多个 data 层的输出 Variable 都是 feed 的输出。

fetch 也是用相同的方式处理。

```

for i, var in enumerate(fetch_list):
    assert isinstance(var, Variable) or isinstance(
        var, six.string_types), (
        "Wrong type for fetch_list[%s]: %s" % (i, type(var)))
    global_block.append_op(
        type='fetch',
        inputs={'X': [var]},
        outputs={'Out': [fetch_var]},
        attrs={'col': i})

```

如代码所示,对于每个需要 fetch 的 Variable,都创建一个 fetch op,以这个 Variable 为输入,然后一个 fetch_var 为输出。

然后返回 clone 的 Program. 其实可以发现 Program 都变了一个,那么那些 Variable 都不是原来那个 Program 上的 Variable 了。而对于 fetch_list 上的变量也是,需要 fetch 的那些变量其实已经不在一开始的那个 Program 上了。Why?因为整套都是根据名字来运行的啊,约等于面向字符串编程。

接下来做的事情是 feed 输入。在这里会把非 core.LoDTensor 的数据转换为 LoDTensor 然后 feed 进去。这是通过这个函数来实现的。(我们后续可以查看一下这个的定义)

```

core.set_feed_variable(scope, cur_feed, feed_var_name, idx)

```

接下来运行的是

```

self._default_executor.run(program.desc, scope, 0, True, True,
    [fetch_var_name])

```

默认的 Executor 是一个 core.Executor 对象。(注意:python 端的 Executor 不是 C++端的 Executor),其 run 方法的定义是

```

.def("run", [](Executor &self, const ProgramDesc &prog, Scope *scope,
    int block_id, bool create_local_scope, bool create_vars,
    const std::vector<std::string> &fetch_vars) {
    pybind11::gil_scoped_release release;
    self.Run(prog, scope, block_id, create_local_scope, create_vars,
        fetch_vars);
});

```

至此。我们几乎明白了整个运行过程。同时这里也需要我们开辟一个新的阅读路线,亦即 python 端的 Executor, Scope, Program,Block,Variable, LoDTensor 分别和 C++端的什么对应。

专题: 静态图下 python 端的 Variable 是如何和 Tensor 联系在一起的?